

Quiz 6: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 6 is Thursday Oct 8, at the end of class. About 15 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

Scope: convolutional networks

Everything in the Week 6 deck, plus Lab 5.

On the quiz:

- What an image is as a tensor, and the layout conventions that cause silent bugs
- Why models normalize their inputs, and why the ImageNet constants are not universal
- Resolution, aliasing, and why you filter before you shrink
- Augmentation: what it is, and when it helps rather than hurts
- The convolution operation, and the output-size arithmetic
- Parameter counts, and why they do not depend on the image size
- Padding, stride, pooling, and the receptive field
- Why small kernels stacked beat one large kernel
- The conv, norm, activation, pool block, and the order of it
- BatchNorm, and the difference between `train()` and `eval()` mode
- 1×1 convolutions and global average pooling
- Residual connections and the problem they fix
- The architecture families, and the one idea each contributed

On the quiz from Lab 5, which was due the day before:

- Predicting a `Conv2d` output shape before running it
- Counting convolution parameters, and what the flatten head would have cost
- Why the CNN beat the MLP at the same parameter count
- Why only the `eval()` curve wobbled
- Why augmentation made things **worse** on Fashion-MNIST, and what changed when the validation images were shifted

Format. Same four parts as Quiz 5.

Part	Kind	Points
A	True or false, 6 questions	6
B	Multiple choice, 4 questions	8
C	Fill in the blank, computational	5
D	Find three bugs in a short script	6

Part D's bugs are **not** the ones from Quiz 5. There is no missing `zero_grad` and no stray softmax this time.

Part 1: Image data

An image is a tensor

A batch is (N, C, H, W) : batch, channels, height, width. PyTorch uses this **CHW** order. Most image libraries hand you **HWC**, and matplotlib wants HWC to display, so `permute` or `transpose` shows up constantly and getting it wrong usually does not raise.

Pixel (r, c) is row r counting from the **top**, column c from the left. The y axis increases downward, which is the opposite of a plot.

Normalization, and the thing to be careful about

Models standardize each channel:

$$x' = \frac{x - \mu_c}{\sigma_c}$$

The ImageNet statistics, $\mu = [0.485, 0.456, 0.406]$ and $\sigma = [0.229, 0.224, 0.225]$, are the ones you meet most often, because most pretrained backbones are ImageNet classifiers.

They are not universal, and the quiz asks about this. Detection models in torchvision take plain $[0, 1]$ and standardize internally. Video models use Kinetics statistics. Optical flow models map to $[-1, 1]$. The rule is not a list to memorize, it is: call `weights.transforms()` and use what it gives you rather than hard-coding constants.

Aliasing

Detail finer than your sampling grid does not just disappear when you shrink an image. It reappears as a coarser pattern that was never in the scene. The fix is to **blur before you shrink**, so the detail that cannot be represented is gone before it can masquerade as something else.

In torchvision, `transforms.Resize(antialias=True)` is the default now and did not used to be.

Augmentation, and when it is worth having

Augmentation applies random **label-preserving** changes to the training images so the model sees a slightly different version each epoch. Two things decide whether it helps.

Does it preserve the label? A vertical flip on digits turns a 6 into a 9 and the label does not follow. That is not augmentation, it is mislabeling.

Does it match the variation your test data has? This is the Lab 5 result and it is worth understanding rather than memorizing. Crop and flip augmentation made Fashion-MNIST validation accuracy **worse** by 3 points, because the images are centered and tightly framed and the footwear classes all point the same way, so the augmentation manufactured images the validation set never contains. Then the same two models were evaluated on shifted validation images and the ranking inverted: the plain model fell 17.75 points while the augmented one fell 6.10.

Augmentation buys robustness to the variation you train for. That is the only thing it buys, and if your data does not have that variation you are paying for nothing.

Part 2: The convolution

Output size

For one spatial axis:

$$W_{\text{out}} = \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1$$

Three cases worth being able to do instantly, on a 28×28 input:

Layer	Out	Why
Conv2d(1, 32, 3, padding=1)	28	padding 1 with kernel 3 is the size-preserving combination
Conv2d(1, 32, 3, padding=0)	26	no padding costs $k - 1$ pixels
Conv2d(1, 32, 5, padding=2, stride=2)	14	$\lfloor 13.5 \rfloor + 1$

The batch axis never changes. The channel axis becomes `out_channels`, whatever that was.

Parameters

$$\text{Conv2d}(C_{\text{in}}, C_{\text{out}}, k) = C_{\text{out}}(C_{\text{in}}k^2 + 1)$$

The image size is not in that formula. Conv2d(3, 16, 3) has 448 parameters on a 28×28 image and 448 on a 224×224 image. That is weight sharing, and it is the first of the two reasons a CNN beats a fully connected network on images. The second is locality: a convolution only ever looks at a small neighborhood, so it does not have to learn from data that neighboring pixels are related.

Compare to a **Linear** reading the same 28×28 image into 32 outputs: $784 \times 32 = 25,120$ weights, used once. The convolution uses 320 and applies them at all 784 positions.

Receptive field

How much of the input one output position depends on. Starting at $r = 1$, $j = 1$ and going forward:

- Conv2d with kernel k : $r \leftarrow r + (k - 1)j$
- MaxPool2d(2): $r \leftarrow r + j$, then $j \leftarrow 2j$

Lab 5 tracked this through Conv(3), Pool(2), Conv(3), Pool(2) and got 3, 4, 8, 10. Ten pixels of twenty-eight after four layers, which is why a network needs depth before any one output can see the whole image.

Small kernels, stacked

Two 3×3 convolutions cover the same 5×5 window as one 5×5 convolution. On 32 channels that is **18,496** parameters against **25,632**, about 28% fewer, and the stack gets a nonlinearity in the middle that the single layer does not. VGG made this standard.

The corollary is Part D of the quiz: **two convolutions with no activation between them are one convolution**. Stacking linear operations gives you a linear operation with a bigger parameter count and no more expressive power.

Pooling

MaxPool2d takes the largest value in each window. It has **no parameters**. A strided convolution does the same downsampling and does have parameters, so it can learn how to downsample; pooling cannot, which is sometimes exactly what you want.

Part 3: Building a CNN

The block, and its order

`Conv2d → BatchNorm2d → ReLU → (MaxPool2d)`

BatchNorm2d takes the **channel count**, not the spatial size, because it keeps one mean and one variance per channel. `BatchNorm2d(32)` has 32 running means, 32 running variances, and 32 each of a learned scale and shift, so 64 learnable parameters.

BatchNorm in `eval()` mode

This is the one runtime trap in the deck and it is on the quiz.

At test time you may have a single image, so there is no batch to compute statistics from. BatchNorm therefore keeps a **running estimate** of the mean and variance during training and uses it in `eval()` mode. That estimate is an exponential moving average, so it **lags** the weights.

While the weights are still moving fast, the lagging estimate is wrong and eval-mode accuracy swings even though nothing is wrong with the model. Lab 5 measured it: over the last nine epochs the `eval()` curve moved **5.9 points** and the `train()` curve moved **1.3**, same weights and same 2,000 images. Decaying the learning rate settles it, because the weights stop moving.

The other half of this: if you forget `model.eval()` at validation, BatchNorm uses the **validation batch's own statistics**. Your score then depends on how the validation set happened to be batched, and one image's prediction depends on the other images sitting next to it.

1×1 convolutions

A 1×1 convolution mixes across **channels** and not across space. Each output pixel is a matrix product of the input channels at that same pixel. It is the cheap way to change channel count, and it is what Inception modules and ResNet bottlenecks use.

Global average pooling against flatten

`AdaptiveAvgPool2d(1)` averages each channel to one number, so a $(N, 96, 7, 7)$ feature map becomes $(N, 96, 1, 1)$ and the classifier reads 96 numbers. Two things this buys:

1. **Parameters.** Lab 5 measured it: `Linear(96*7*7, 10)` is **47,050** parameters against **970** for `Linear(96, 10)`, a factor of 48.5, which would have been about a fifth of the whole network.
2. **Any input size.** A flatten head has to be told $96 \times 7 \times 7$, and that number is wrong as soon as the input size changes. The pooled head does not care.

Every architecture after VGG uses global average pooling.

Residual connections

$$y = \mathcal{F}(x) + x$$

They exist to fix the **degradation problem**, and the precise statement matters. A 56-layer plain network had higher error than a 20-layer one **on the training set**. That is not overfitting. The deeper network could represent whatever the shallow one did, by setting the extra layers to the identity, and it could not find that solution by gradient descent. The skip connection makes the identity the default, so the layers only have to learn the difference from it.

CNN against MLP, measured

Lab 5's comparison, which you ran yourself: same 12,000 Fashion-MNIST images, same AdamW, same cosine schedule, same 18 epochs.

Model	Parameters	Validation accuracy
Linear(784,256) then Linear(256,10)	203,530	87.50%
The CNN	204,970	92.70%

Read it as an error rate: 12.5% down to 7.3%, so **four of every ten mistakes are gone** at the same parameter count. What bought that is locality and weight sharing, not capacity.

Part 4: Architectures

You will not be asked for dates or layer counts. You will be asked what an architecture **contributed**.

Architecture	The one idea
LeNet-5	Convolutions and pooling work for images at all
AlexNet	Depth plus ReLU plus GPUs plus dropout, at ImageNet scale
VGG	Only 3×3 kernels, stacked; depth as the one variable
Inception	Several kernel sizes in parallel, with 1×1 to keep it cheap
ResNet	Skip connections, which fix the degradation problem
DenseNet	Concatenate every earlier feature map instead of adding
MobileNet	Depthwise separable convolution: factor a conv into spatial and channel parts

Grouped convolution splits the channels into g groups and convolves within each, cutting the parameter count by about g . Depthwise separable convolution is the extreme case, one group per channel, followed by a 1×1 to mix channels back.

Part 5: What to be able to do without thinking

1. **Output shape.** Given a `Conv2d` or a pool and an input shape, give the output shape. Four of the 25 points are this, directly.
2. **Parameter count.** For a `Conv2d` and for a `Linear`. Two more points.
3. **Receptive field.** Run the recursion through a few layers.
4. **Spot a missing activation.** Two convolutions in a row is one convolution.
5. **Spot a missing `model.eval()`,** and say the BatchNorm reason rather than the dropout reason.
6. **Say whether an augmentation preserves the label,** given the dataset.

Checklist

Before the quiz, make sure you can answer these without looking:

- What does (N, C, H, W) mean, and which axis does a convolution's `out_channels` set?
- Why does a convolution's parameter count not involve H and W ?
- What padding do you need with a 3×3 kernel to keep the size?
- What does `BatchNorm2d(64)` actually store, and how much of it is learned?
- What is different about BatchNorm between `train()` and `eval()`?
- What does a 1×1 convolution do that a 3×3 does not do better?
- Two reasons to use global average pooling instead of flatten.
- What was the degradation problem, and what word in the statement is doing the work?
- Why did the CNN beat the MLP in Lab 5 at the same parameter count?
- Name an augmentation that would break the label on a dataset of digits, and one that would not.